

Algorithmique 1

4. Algorithmes de tri

Johan Thapper
2023–2024



Classement

```
$ ls -lh
```

```
...
```

```
drwxr-xr-x  9 thapper  admin  306B Dec 17 16:48 urllib
-rwxr-xr-x  1 thapper  admin  6.6K Dec 17 16:48 uu.py
-rw-r--r--  1 thapper  admin   23K Dec 17 16:48 uuid.py
drwxr-xr-x  6 thapper  admin  204B Dec 17 16:48 venv
-rw-r--r--  1 thapper  admin   15K Dec 17 16:48 warnings.py
-rw-r--r--  1 thapper  admin   17K Dec 17 16:48 wave.py
-rw-r--r--  1 thapper  admin   19K Dec 17 16:48 weakref.py
-rwxr-xr-x  1 thapper  admin   21K Dec 17 16:48 webbrowser.py
drwxr-xr-x  9 thapper  admin  306B Dec 17 16:48 wsgiref
-rw-r--r--  1 thapper  admin   5.8K Dec 17 16:48 xdrlib.py
drwxr-xr-x  8 thapper  admin  272B Dec 17 16:48 xml
-rw-r--r--  1 thapper  admin   7.0K
-rw-r--r--  1 thapper  admin   72K
```

clés



```
$ ls -Slhr
```

```
...
```

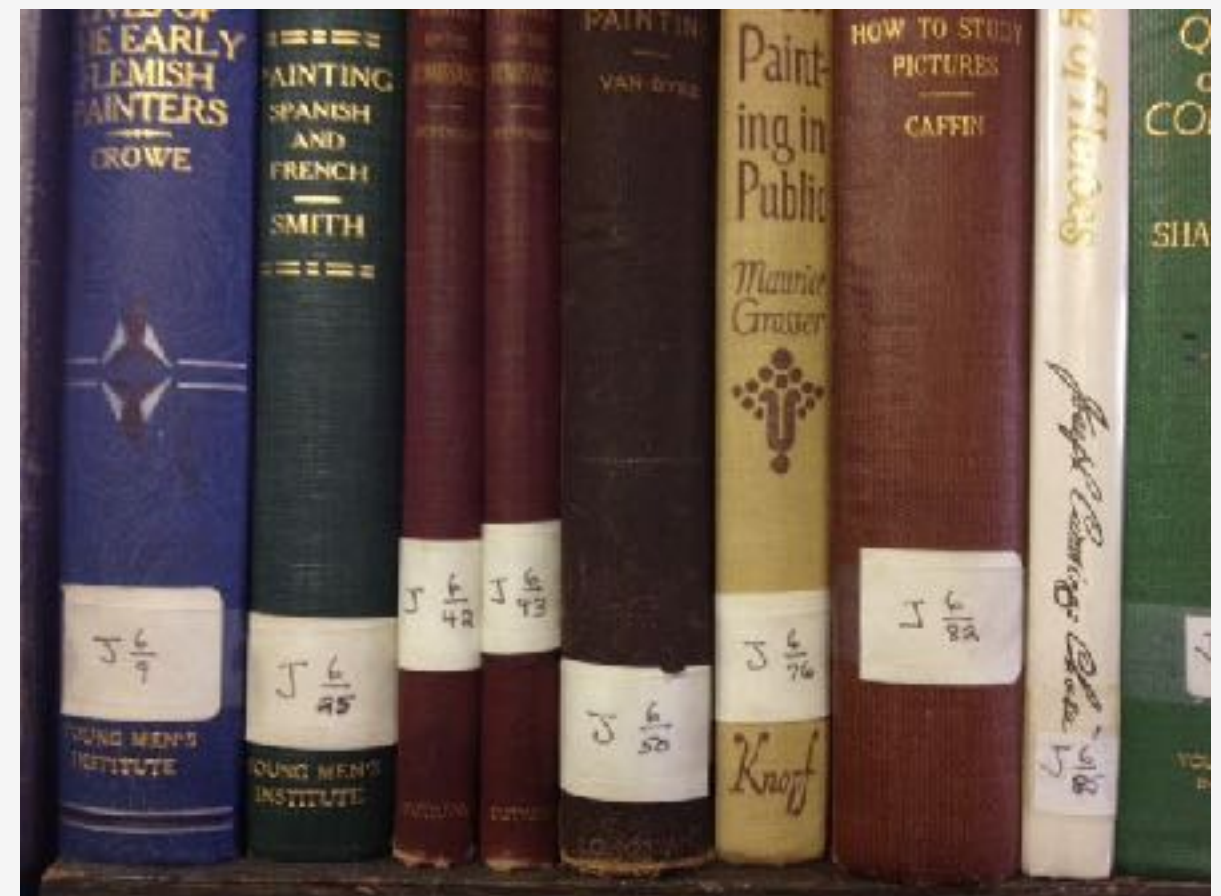
```
-rw-r--r--  1 thapper  admin   73K Dec 17 16:48 locale.py
-rw-r--r--  1 thapper  admin   74K Dec 17 16:48 ipaddress.py
-rw-r--r--  1 thapper  admin   74K Dec 17 16:48 datetime.py
-rw-r--r--  1 thapper  admin   77K Dec 17 16:48 mailbox.py
-rw-r--r--  1 thapper  admin   82K Dec 17 16:48 difflib.py
-rw-r--r--  1 thapper  admin   86K Dec 17 16:48 _pyio.py
-rw-r--r--  1 thapper  admin   88K Dec 17 16:48 argparse.py
-rw-r--r--  1 thapper  admin   90K Dec 17 16:48 pickletools.py
-rwxr-xr-x  1 thapper  admin   91K Dec 17 16:48 tarfile.py
-rwxr-xr-x  1 thapper  admin  101K Dec 17 16:48 pydoc.py
-rw-r--r--  1 thapper  admin  102K Dec 17 16:48 doctest.py
-rw-r--r--  1 thapper  admin  111K Dec 17 16:48 inspect.py
-rw-r--r--  1 thapper  admin  140K Dec 17 16:48 turtle.py
-rw-r--r--  1 thapper  admin  222K Dec 17 16:48 _pydecimal.py
```

données →

Exemples



- ▶ Clés : mots, codes de classification, dates, entiers, ...
- ▶ Pourquoi classer ?
 - ❖ accès plus rapide aux données
 - ❖ est-ce qu'une clé est présente ?
 - ❖ y a-t-il des doublons ?



Simplifier et abstraire

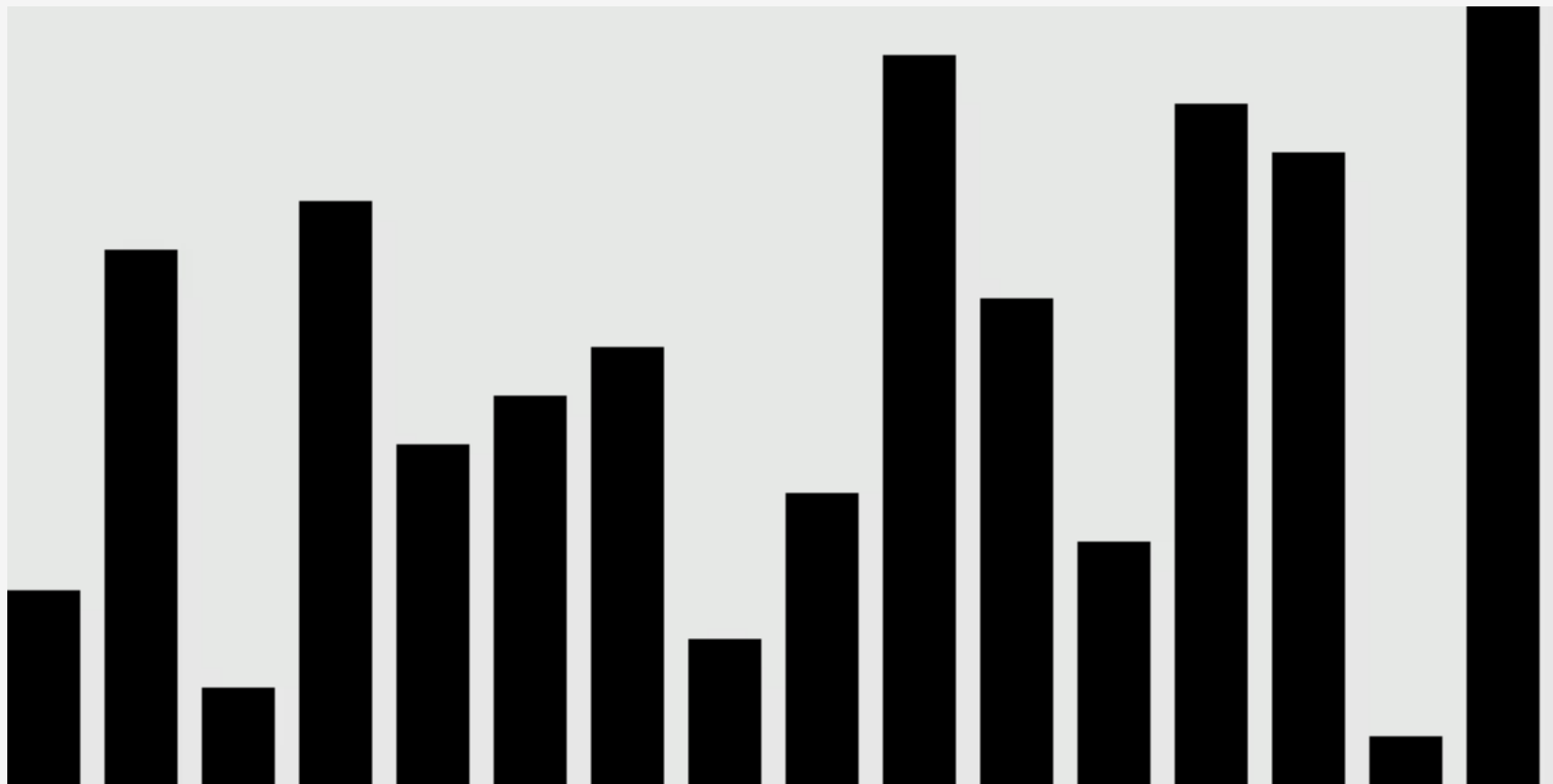
- ▶ On suppose que les clés sont des entiers et qu'il n'y a pas de données associées
- ▶ On cherche à classer les clés en ordre croissant (non-strict), $\leq$
- ▶ On utilise dans un premier temps exclusivement deux fonctions primitives pour accéder et modifier le tableau :
 - ❖ `less(a, b)` — renvoie 1 si $a < b$, 0 sinon
 - ❖ `swap(&a, &b)` — échange les valeurs de `a` et `b`
- ▶ Pour nos clés, ces fonctions deviennent :

```
int less(int a, int b) {  
    return a < b;  
}
```

```
void swap(int *a, int *b) {  
    int c = *a; *a = *b; *b = c;  
}
```

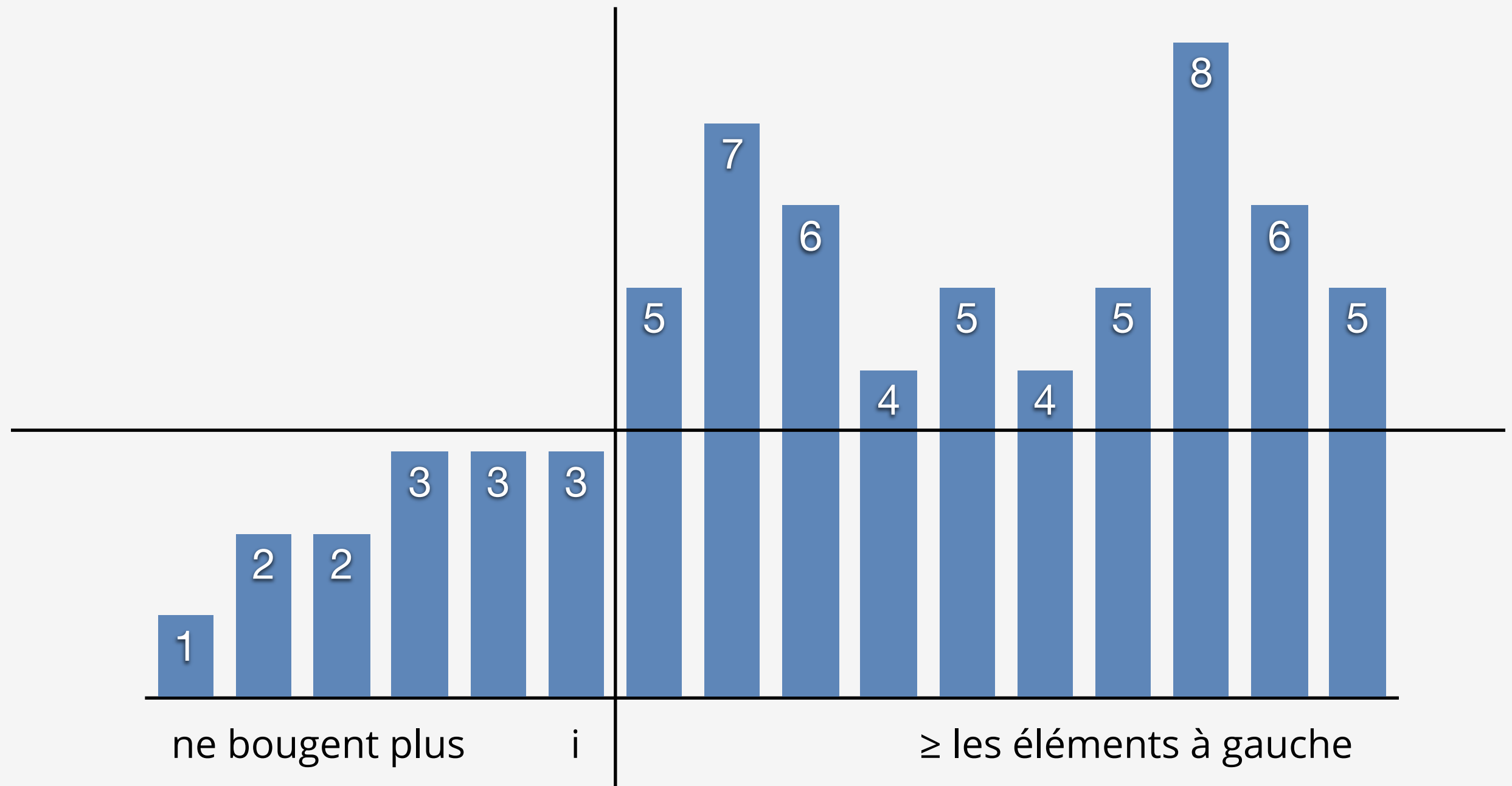
Tri par sélection

- ▶ parcourir le tableau
- ▶ itération i : chercher l'indice min du plus petit élément (dans la partie noir)
 - ❖ échanger $t[i]$ et $t[\text{min}]$



Tri par sélection

- ▶ Après l'itération i



Implantation

parcours →

échange →

```
void selection_sort(int t[], int size) {  
    int i;  
    for (i = 0; i < size; i++) {  
        int min = i;  
        int j;  
        for (j = i+1; j < size; j++)  
            if (less(t[j], t[min]))  
                min = j;  
        swap(&t[i], &t[min]);  
    }  
}
```

← find min

Exemple : déroulement tri par sélection

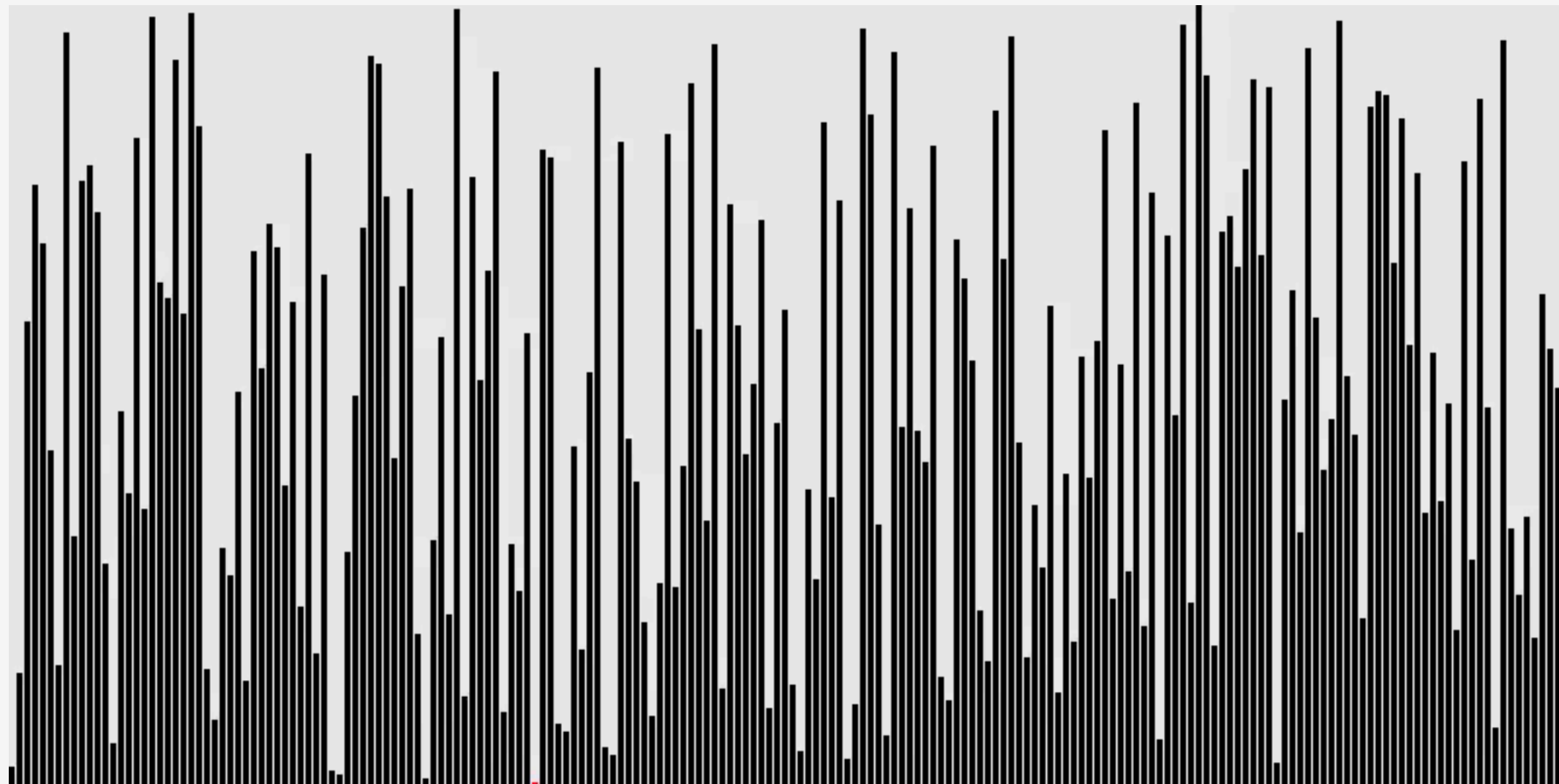
t		7	4	3	2	3	6	7	9	4	3
i	min	0	1	2	3	4	5	6	7	8	9
0	3	7	4	3	2	3	6	7	9	4	3
1	2	2	4	3	7	3	6	7	9	4	3
2	4	2	3	4	7	3	6	7	9	4	3
3	9	2	3	3	7	4	6	7	9	4	3
4	4	2	3	3	3	4	6	7	9	4	7
5	8	2	3	3	3	4	6	7	9	4	7
6	8	2	3	3	3	4	4	7	9	6	7
7	8	2	3	3	3	4	4	6	9	7	7
8	9	2	3	3	3	4	4	6	7	9	7
9	9	2	3	3	3	4	4	6	7	7	9
		2	3	3	3	4	4	6	7	7	9

avant l'échange à l'itération i

t[min]
 ↓
 2 3 3 3 4 4 | 7 9 6 7
 ne bougent plus | comparés

*le nombre de comparaisons ?
le nombre d'échanges ?*

Tri par sélection $N = 200$



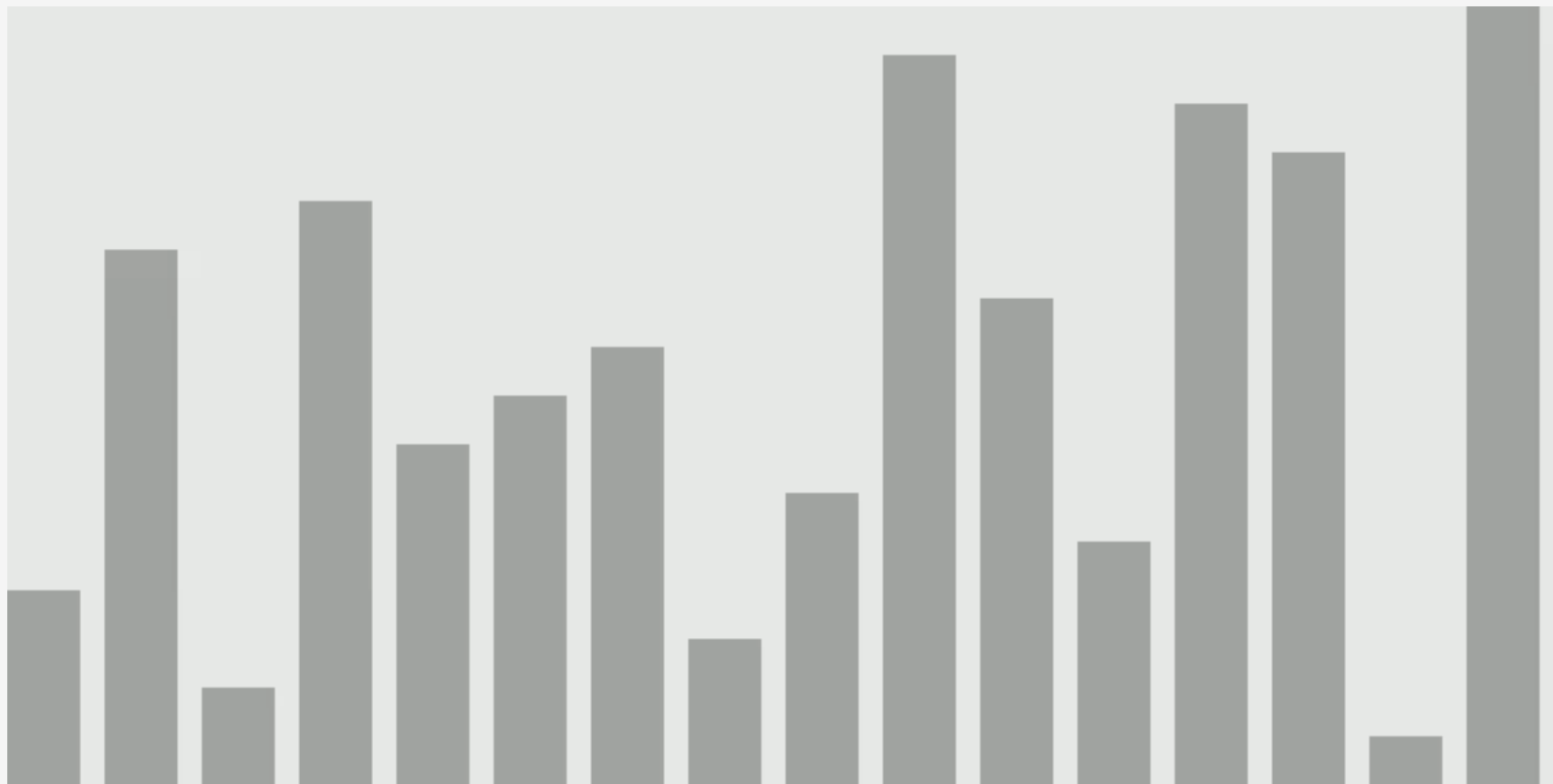
Petites améliorations

```
void selection_sort(int t[], int size) {
    int i;
    for (i = 0; i < size; i++) {
        int min = i;
        int j;
        for (j = i+1; j < size; j++)
            if (less(t[j], t[min]))
                min = j;
        swap(&t[i], &t[min]);
    }
}
```

```
void selection_sort(int t[], int size) {
    int i;
    for (i = 0; i < size-1; i++) {
        int min = i;
        int j;
        for (j = i+1; j < size; j++)
            if (less(t[j], t[min]))
                min = j;
        if (i != min)
            swap(&t[i], &t[min]);
    }
}
```

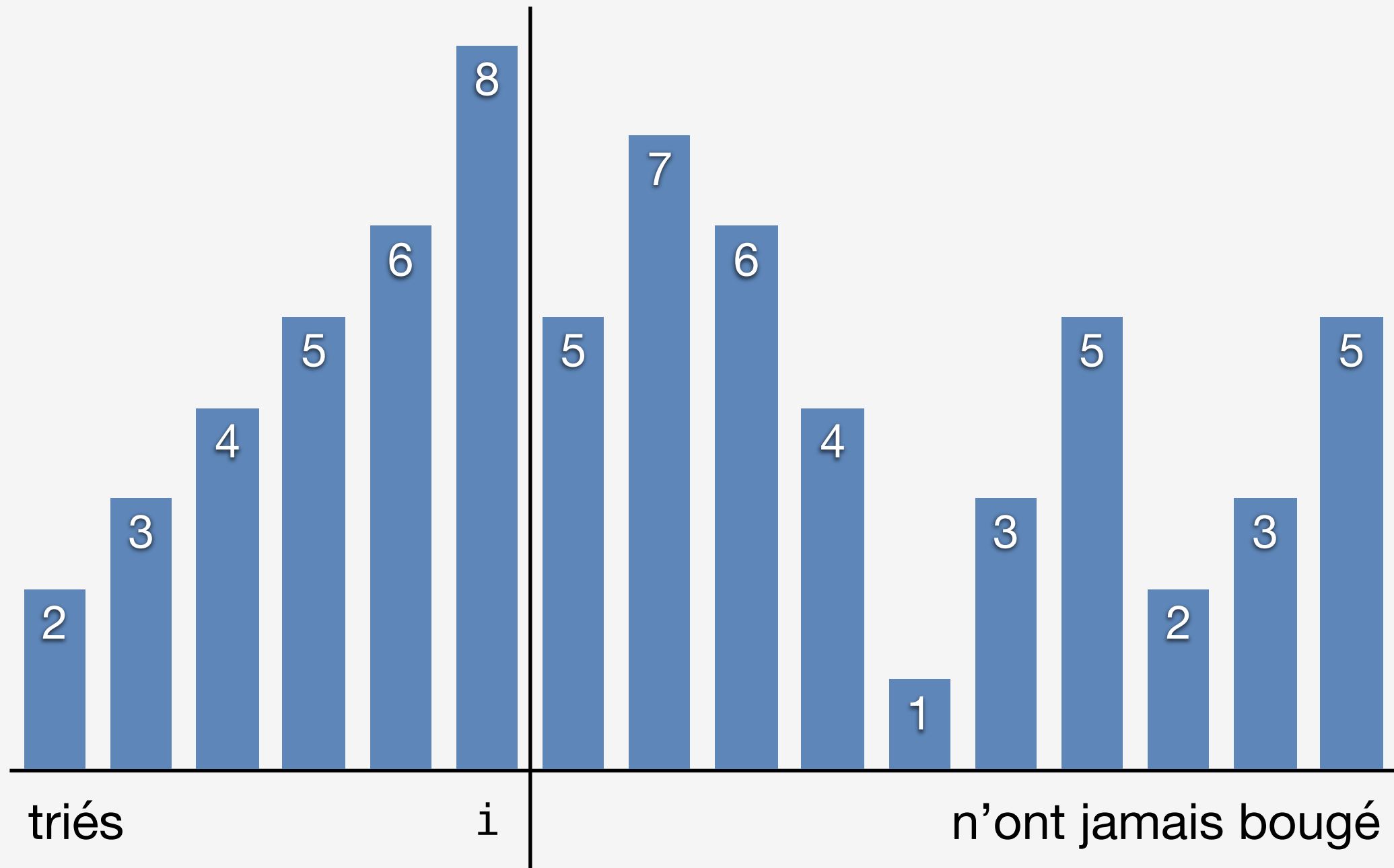
Tri par insertion

- ▶ parcourir le tableau
- ▶ à l'itération i : faire descendre $t[i]$ à sa propre position dans la partie déjà triée (insertion dans un tableau trié)



Tri par insertion

- ▶ Après l'itération i



Implantation

parcours →

```
void insertion_sort(int t[], int size) {  
    int i;  
    for (i = 0; i < size; i++) {  
        int j = i;  
        while (j > 0 && less(t[j], t[j-1])) {  
            swap(&t[j-1], &t[j]);  
            j--;  
        }  
    }  
}
```

↑
insert_sorted(t, 0, i-1, t[i])
vue au td et tp

Exemple : déroulement tri par insertion

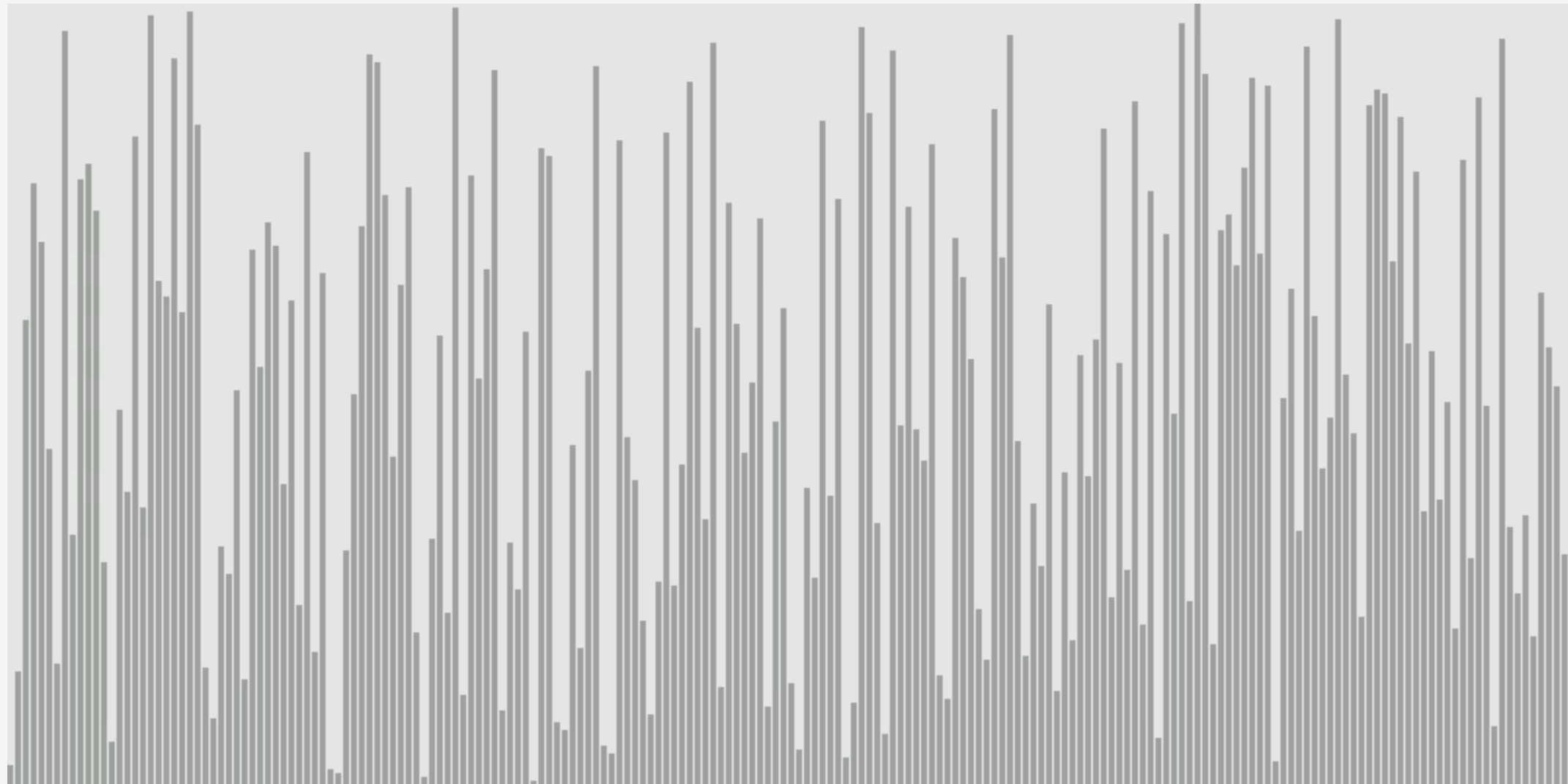
t		7	4	3	2	3	6	7	9	4	3
i	j	0	1	2	3	4	5	6	7	8	9
0	0	7	4	3	2	3	6	7	9	4	3
1	0	4	7	3	2	3	6	7	9	4	3
2	0	3	4	7	2	3	6	7	9	4	3
3	0	2	3	4	7	3	6	7	9	4	3
4	2	2	3	3	4	7	6	7	9	4	3
5	4	2	3	3	4	6	7	7	9	4	3
6	6	2	3	3	4	6	7	7	9	4	3
7	7	2	3	3	4	6	7	7	9	4	3
8	4	2	3	3	4	4	6	7	7	9	3
9	3	2	3	3	3	4	4	6	7	7	9
		2	3	3	3	4	4	6	7	7	9

après l'insertion à l'itération i



*le nombre de comparaisons ?
le nombre d'échanges ?*

Tri par insertion $N = 200$

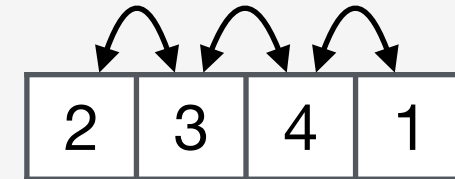


Amélioration : déplacements

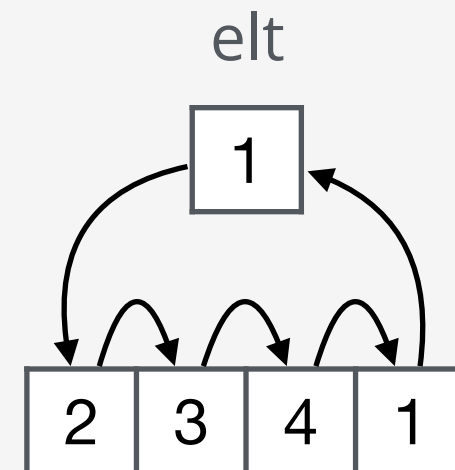
```
void insertion_sort(int t[], int size) {  
    int i;  
    for (i = 0; i < size; i++) {  
        int j = i;  
        while (j > 0 && less(t[j], t[j-1])) {  
            swap(&t[j-1], &t[j]);  
            j--;  
        }  
    }  
}
```

```
void swap(int *a, int *b) {  
    int c = *a; *a = *b; *b = c;  
}
```

3 x swap = 9 affectations



```
void insertion_sort(int t[], int size) {  
    int i;  
    for (i = 0; i < size; i++) {  
        int j = i;  
        int elt = t[j];  
        while (j > 0 && less(elt, t[j-1])) {  
            t[j] = t[j-1];  
            j--;  
        }  
        t[j] = elt;  
    }  
}
```

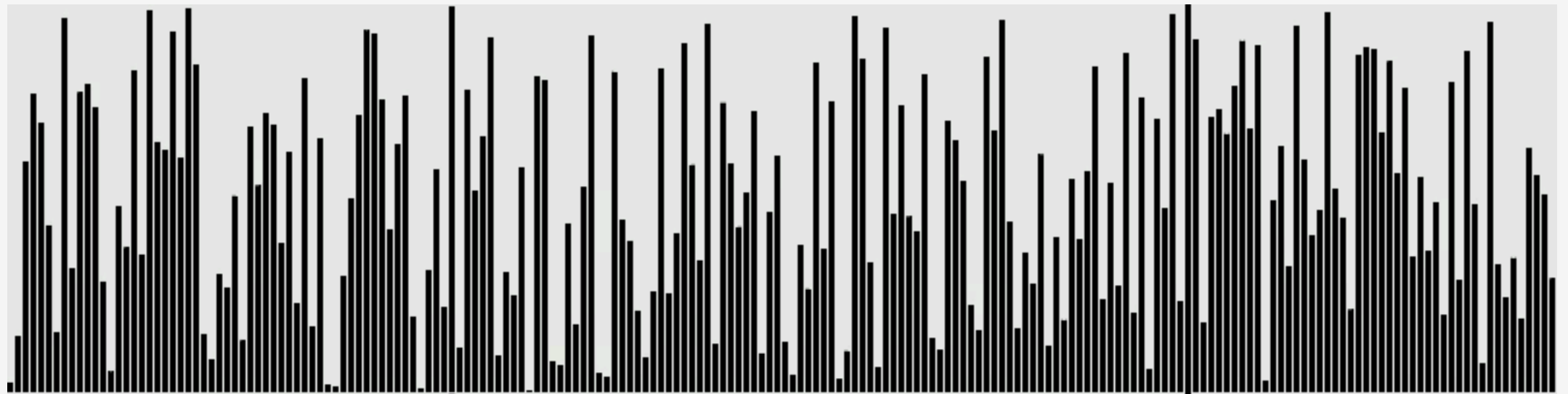


5 affectations

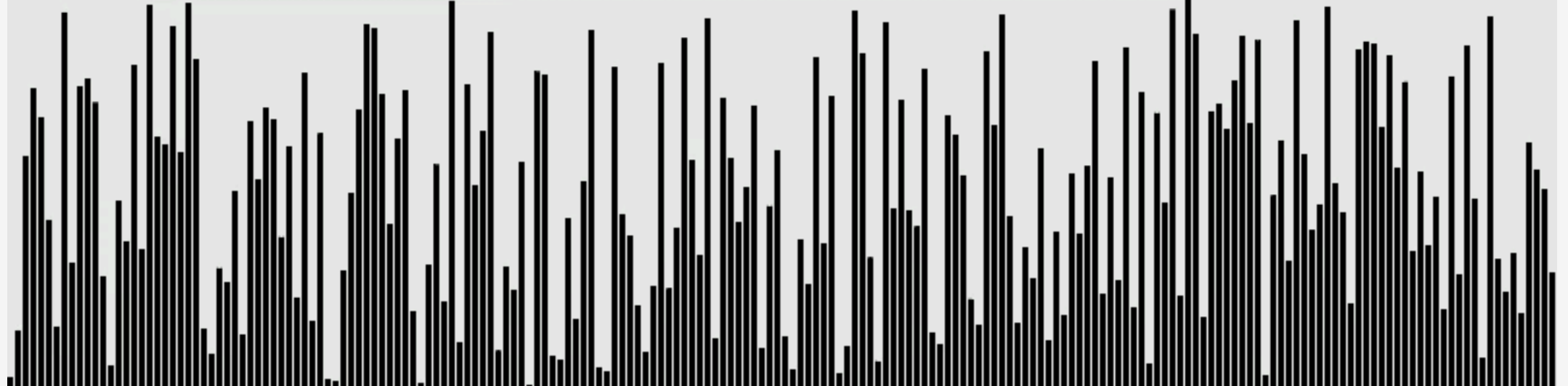
Comparaison $N = 200$

la boucle extérieure synchronisée

insertion



sélection



Comparaison

	tri par insertion	tri par sélection
comparaisons pire cas	$n(n-1)/2$	$n(n-1)/2$
échanges pire cas	$n(n-1)/2$	$n-1$
comparaisons meilleur cas	$n-1$	$n(n-1)/2$
échanges meilleur cas	0	0

- ▶ Tri par insertion
 - ❖ efficace pour des petits tableaux et des tableaux partiellement triés
- ▶ Tri par sélection
 - ❖ efficace si les échanges coûtent cher